Système et Réseaux
CM9 : sérialisation, TCP, `poll`

Pierre Rousselin

Université Sorbonne Paris Nord
L2 informatique et double-licence

12 avril 2026

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

Règles d'alignement, exemple

Imaginons une machine dans laquelle

- la mémoire est adressée en octet (*byte addressable*) ;
- un entier « normal » est stocké sur 4 octets ;
- le processeur récupère les octets 4 par 4 (on parle de « mot mémoire de 4 octets »), par exemples :
  - octets numéros 0, 1, 2 et 3 ;
  - octets numéros 1000, 1001, 1002 et 1003.

Règles d'alignement, exemple

Imaginons une machine dans laquelle

- la mémoire est adressée en octet (*byte addressable*) ;
- un entier « normal » est stocké sur 4 octets ;
- le processeur récupère les octets 4 par 4 (on parle de « mot mémoire de 4 octets »), par exemples :
  - octets numéros 0, 1, 2 et 3 ;
  - octets numéros 1000, 1001, 1002 et 1003.

Que faire lorsque l'instruction est « récupérer l'entier qui commence à l'adresse 998 (qui n'est pas multiple de 4) ?

Règles d'alignement, exemple

Imaginons une machine dans laquelle

- la mémoire est adressée en octet (*byte addressable*) ;
- un entier « normal » est stocké sur 4 octets ;
- le processeur récupère les octets 4 par 4 (on parle de « mot mémoire de 4 octets »), par exemples :
  - octets numéros 0, 1, 2 et 3 ;
  - octets numéros 1000, 1001, 1002 et 1003.

Que faire lorsque l'instruction est « récupérer l'entier qui commence à l'adresse 998 (qui n'est pas multiple de 4) ?

- Sur certaines machines, planter !
- Sur d'autres, récupérer les octets 996 à 999 puis les octets 1000 à 1003
- La donnée qui nous intéresse est sur les octets 998 à 1001.
- On a perdu une instruction (potentiellement coûteuse) par rapport au cas où l'entier aurait commencé à l'adresse 1000.

Règles d'alignement, exemple

- Dans l'exemple (réaliste) précédent, on dit que les entiers sont *alignés sur les adresses multiples de* 4.
- Il pourrait en être autrement sur une autre machine.
- Et chaque compilateur gère à sa façon les contraintes d'alignement.

Exemple avec `gcc`, sans option, sur `x86_64` ;

- pas de contrainte sur `char` (ou tout autre type sur 8 bits comme `uint8_t` ou `int8_t`) ;
- `short` (et `uint16_t` et `int16_t`) alignés sur multiples de 2 ;
- `int` (et `int32_t`), `unsigned` (et `uint32_t`), `float` alignés sur multiples de 4 ;
- `double` et tout pointeur sur multiples de 8.

Remplissage

- Pour respecter les contraintes d'alignement, un compilateur peut remplir la mémoire avec des octets supplémentaires.
- C'est le *remplissage* ou *padding*.
- Peut avoir lieu dans la pile, dans la zone de données et dans les structures.

Exemple dans la zone de données

```c
#include <stdio.h>
int x = 3;
char c = 'I';
int y = 67;
int main(void)
{
    printf("%p\n", &x);
    printf("%p\n", &c);
    printf("%p\n", &y);
    return 0;
}
```

Exemple dans la zone de données

```c
#include <stdio.h>
int x = 3;
char c = 'I';
int y = 67;
int main(void)
{
    printf("%p\n", &x);
    printf("%p\n", &c);
    printf("%p\n", &y);
    return 0;
}
```

```
$ gcc padding_1.c
$ ./a.out
0x404024
0x404028
0x40402c
```

Exemple dans la zone de données

```c
#include <stdio.h>
int x = 3;
char c = 'I';
int y = 67;
int main(void)
{
    printf("%p\n", &x);
    printf("%p\n", &c);
    printf("%p\n", &y);
    return 0;
}
```

```
$ gcc padding_1.c
$ ./a.out
0x404024
0x404028
0x40402c
```

Les octets `0x404029`, `0x40402a`, `0x40402b` ne servent qu'au *padding*, de sorte que l'entier `y` respecte les contraintes d'alignements.

Exemple dans la pile

```c
#include <stdio.h>
int main(void)
{
    int x = 3;
    char c = 'I';
    int y = 67;
    printf("%p\n", &x);
    printf("%p\n", &c);
    printf("%p\n", &y);
    return 0;
}
```

Exemple dans la pile

```c
#include <stdio.h>
int main(void)
{
    int x = 3;
    char c = 'I';
    int y = 67;
    printf("%p\n", &x);
    printf("%p\n", &c);
    printf("%p\n", &y);
    return 0;
}
```

```
$ gcc padding_2.c
$ ./a.out
0x7ffd8073904c
0x7ffd8073904b
0x7ffd80739044
```

Exemple dans la pile

```c
#include <stdio.h>
int main(void)
{
    int x = 3;
    char c = 'I';
    int y = 67;
    printf("%p\n", &x);
    printf("%p\n", &c);
    printf("%p\n", &y);
    return 0;
}
```

```
$ gcc padding_2.c
$ ./a.out
0x7ffd8073904c
0x7ffd8073904b
0x7ffd80739044
```

- `x` occupe les cases `4c`, `4d`, `4e` et `4f` ;
- `c` occupe la case `4b` ;
- *padding* sur `4a`, `49` et `48` ;
- `y` occupe les cases `44`, `45`, `46` et `47`.

Exemple avec une structure

```
struct foo {
    uint32_t n;
    char c;
    uint32_t m;
};
int main(void)
{
    printf("%lu\n", sizeof(struct foo));
    struct foo x = { .n = 127, .c = 'A', .m = 564 };
    printf("%p\n", &x.n);
    printf("%p\n", &x.c);
    printf("%p\n", &x.m);
    return 0;
}
```

Exemple avec une structure

```
struct foo {
    uint32_t n;
    char c;
    uint32_t m;
};
int main(void)
{
    printf("%lu\n", sizeof(struct foo));
    struct foo x = { .n = 127, .c = 'A', .m = 564 };
    printf("%p\n", &x.n);
    printf("%p\n", &x.c);
    printf("%p\n", &x.m);
    return 0;
}
```

```
$ gcc -Wall struct_padding.c
$ ./a.out
12
0x7ffd8158f730
0x7ffd8158f734
0x7ffd8158f738
```

Exemple avec une structure

Notre structure est en faite équivalente en mémoire à

```
struct foo {
    uint32_t n;
    char c;
    char padding[3];
    uint32_t m;
};
```

avec `padding` non utilisé.

Parenthèse : fonction pour afficher des octets

```
void print_bytes(void *buf, size_t nbytes);
```

Parenthèse : fonction pour afficher des octets

```
void print_bytes(void *buf, size_t nbytes);

void print_bytes(void *buf, size_t nbytes)
{
    uint8_t *p;
    for (p = buf; nbytes > 0; ++p, --nbytes) {
        printf("%02x", *p);
    }
}
```

Parenthèse : fonction pour afficher des octets

```
void print_bytes(void *buf, size_t nbytes);

void print_bytes(void *buf, size_t nbytes)
{
    uint8_t *p;
    for (p = buf; nbytes > 0; ++p, --nbytes) {
        printf("%02x", *p);
    }
}

struct foo x = { .n = 127, .c = 'A', .m = 564 };
/* On peut aller voir octet par octet : */
print_bytes(&x, sizeof(x));
```

Parenthèse : fonction pour afficher des octets

```
void print_bytes(void *buf, size_t nbytes);

void print_bytes(void *buf, size_t nbytes)
{
    uint8_t *p;
    for (p = buf; nbytes > 0; ++p, --nbytes) {
        printf("%02x", *p);
    }
}

struct foo x = { .n = 127, .c = 'A', .m = 564 };
/* On peut aller voir octet par octet : */
print_bytes(&x, sizeof(x));

7f 00 00 00 41 00 00 00 34 02 00 00
```

Parenthèse : fonction pour afficher des octets

```
void print_bytes(void *buf, size_t nbytes);

void print_bytes(void *buf, size_t nbytes)
{
    uint8_t *p;
    for (p = buf; nbytes > 0; ++p, --nbytes) {
        printf("%02x", *p);
    }
}

struct foo x = { .n = 127, .c = 'A', .m = 564 };
/* On peut aller voir octet par octet : */
print_bytes(&x, sizeof(x));

7f 00 00 00 41 00 00 00 34 02 00 00
```

(Noter au passage le petit-boutisme de `x86_64`).

Structures et réseau ne s'entendent pas bien

- On ne peut faire aucune hypothèse sur l'alignement.
- On ne peut faire aucune hypothèse sur le placement des champs dans les octets.
- Donc on n'envoie jamais de `struct` dans le réseau.

Structures et réseau ne s'entendent pas bien

- On ne peut faire aucune hypothèse sur l'alignement.
- On ne peut faire aucune hypothèse sur le placement des champs dans les octets.
- Donc on n'envoie jamais de `struct` dans le réseau.
- Mais les `struct`, on ne peut pas vraiment s'en passer...
- Alors que faire ?

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

Comme toujours, la réponse est dans le protocole applicatif

- On doit donc décider nous-même de l'ordre dans lequel les données sont transmises.
- Et transformer les `struct` en suite d'octets, transportable et non ambigüe.
- C'est ce qu'on appelle la sérialisation (*serialization*, *marshalling*) des données.

```
/* Mettre les données dans un tableau d'octets
 * Nécessite d'être très clair sur l'ordre des octets (protocole !)
 * Ici, choisissons
 * 1) n sur 4 octets (bits  0 à 31) en network byte order
 * 2) m sur 4 octets (bits 32 à 63) en network byte order
 * 3) c sur 1 octet  (bits 64 à 71) */
uint8_t donnees[2 * sizeof(uint32_t) + 1];
*((uint32_t *) donnees) = htonl(x.n);
*((uint32_t *) donnees + 1) = htonl(x.m);
*(donnees + 2 * sizeof(uint32_t)) = x.c;
/* on peut envoyer donnees à un autre processus */
```

Désérialisation

Chez le processus qui lit. Le protocole applicatif doit être assez précis pour convertir de nouveau la suite d'octets en un type plus structuré. C'est la désérialisation (*deserialization*, *unmarshalling*).

```
uint8_t donnees[2 * sizeof(uint32_t) + 1];
read(sock, donnees, sizeof(donnees));
struct foo x;
x.n = ntohl(*((uint32_t *) donnees));
x.m = ntohl(*(((uint32_t *) donnees) + 1));
x.c = *(donnees + 2 * sizeof(uint32));
```

Désérialisation

Chez le processus qui lit. Le protocole applicatif doit être assez précis pour convertir de nouveau la suite d'octets en un type plus structuré. C'est la désérialisation (*deserialization*, *unmarshalling*).

```
uint8_t donnees[2 * sizeof(uint32_t) + 1];
read(sock, donnees, sizeof(donnees));
struct foo x;
x.n = ntohl(*((uint32_t *) donnees));
x.m = ntohl(*(((uint32_t *) donnees) + 1));
x.c = *(donnees + 2 * sizeof(uint32));
```

Bien sûr, on écrit des fonctions pour la sérialisation et la désérialisation de la structure.

Sérialisation/Désérialisation

```
void htonfoo(const struct foo *src, uint8_t *dest)
{
    *((uint32_t *) dest) = htonl(src->n);
    *((uint32_t *) dest + 1) = htonl(src->m);
    *(dest + 2 * sizeof(uint32_t)) = src->c;
}
void ntohfoo(const uint8_t *src, struct foo *dest) {
    dest->n = ntohl(*((uint32_t *) src));
    dest->m = ntohl(*((uint32_t *) src + 1));
    dest->c = *(src + 2 * sizeof(uint32_t));
}
```

Sérialisation/Désérialisation

Utilisation :

```
int main(void)
{
    /* socket, etc */
    struct foo x = { .n = 127, .c = 'A', .m = 564 };
    uint8_t donnees[2 * sizeof(uint32_t) + 1];
    /* Écriture dans le réseau d'une struct */
    htonfoo(&x, donnees);
    write(sock, donnees, sizeof(donnees));
    /* ... */
    /* Lecture dans le réseau d'une struct */
    read(sock, donnees, sizeof(donnees));
    ntohfoo(donnees, &x);
    /* ... */
}
```

Parenthèse : memcpy (<string.h>)

```
NOM
       memcpy - Copier une zone mémoire
SYNOPSIS
       #include <string.h>
       void *memcpy(void *dest, const void *src, size_t n);
```

Implémentation de htonfoo et ntohfoo avec memcpy :

```
void htonfoo_memcpy(const struct foo *src, uint8_t *dest)
{
    /* TODO: Gérer l'endianness... */
    memcpy(dest, &src->n, 4);
    memcpy(dest + 4, &src->m, 4);
    memcpy(dest + 8, &src->c, 1);
}
void ntohfoo_memcpy(const uint8_t *src, struct foo *dest)
{
    /* TODO: Gérer l'endianness... */
    memcpy(&dest->n, src, 4);
    memcpy(&dest->m, src + 4, 4);
    memcpy(&dest->c, src + 8, 1);
}
```

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

TCP, rôle

TCP signifie *Transmission Control Protocol*.

- protocole de la couche transport donc mis en œuvre par les systèmes d'exploitation
- numéro de port, comme pour UDP, un processus communique avec un processus
- les messages, arbitrairement longs, sont séparés en segments TCP
- le rôle de TCP est de fiabiliser les échanges donc de :
  - réémettre les paquets manquants ou corrompus
  - s'assurer que les segments dans le bon ordre
  - jeter les segments en double (doublons, *duplicates*) ou corrompus
- Ce sont les systèmes d'exploitation (SE) de chaque hôte qui s'échangent des informations et essaient très dur d'accomplir ces missions.

TCP, 3 étapes

- La connexion (les 3 poignées de main) :
  - le SE d'un client souhaitant débuter des échanges en TCP avec un serveur envoie un premier segment au SE de ce serveur lui signifiant son envie d'échanger avec lui
  - le SE du serveur serveur lui répond « ok » en lui transmettant certaines informations
  - le SE du client confirme qu'il a reçu le « ok »
- Les échanges : les SE des deux hôtes se donnent beaucoup de mal pour que tout se passe bien, mais du côté des processus tout se passe comme s'ils se transmettaient des flux d'octets.
- La fermeture de la connexion, le plus souvent 4 poignées de mains :
  - L'un des processus dit « FIN ! » et attend la confirmation de la réception de l'autre.
  - L'autre processus dit « Bien reçu ! » puis « FIN ! »
  - Avant de fermer la connexion, le premier processus dit « Bien reçu ! »

Quelques éléments du fonctionnment de TCP (1)

Par manque de temps, nous restons à un niveau très vague. TCP est un protocole vraiment très compliqué, on pourrait y passer de nombreuses semaines !

- Chaque SE maintient et transmet 2 numéros :
  - le numéro d'aquittement (`ack`) qui contient l'information : « j'ai reçu tous les octets jusqu'à l'octet numéro x » ;
  - le numéro de séquence (`seq`) qui contient l'information : « j'ai envoyé tous les octets jusqu'à l'octet numéro y ».
- Les segments TCP contiennent donc, avec les données, les informations :
  - « bien reçu jusqu'à l'octet x ! »
  - « chez moi, j'ai transmis jusqu'à l'octet y ! »
- Un émetteur qui ne reçoit pas l'aquittement (« bien reçu ! ») au bout d'un certain temps (constamment recalculé) appelé RTO (*retransmission timeout*) réémet les segments non aquittés.

Quelques éléments du fonctionnement de TCP (2)

- Un émetteur a également une fenêtre glissante (*sliding window*), il n'envoie pas plus de n segments avant d'avoir reçu l'aquittement des segments précédents.
- La taille de la fenêtre peut varier pendant les échanges : plus les acquittements sont reçus rapidement, plus l'émetteur pourra envoyer de segments, et inversement.
- Ceci permet une forme de *contrôle de congestion*.
- Il ne faudrait pas qu'un émetteur envoie des segments beaucoup plus vite que la possibilité pour le récepteur de les traiter et les aquitter.

Quelques éléments du fonctionnement de TCP (3)

- Les numéros de séquences permettent de réordonner, si besoin, les segments.
- Une somme de contrôle sur tout le segment permet de détecter des corruptions de segment : un paquet corrompu est simplement ignoré et non aquitté (il sera donc réémis).
- Un segment perdu n'est pas reçu donc n'est pas aquitté et sera donc réémis.

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

sockets TCP

De ce qui précède, on retient :

- il doit y avoir des échanges de segments (gérés par les SE) avant les échanges de données
- la connexion a aussi une fin : il y a des échanges de segments après les échanges de données.

Côté client : connect

- Le client crée une *socket* TCP (par exemple IPv4) :

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
if (fd < 0) { /* ... */ }
```

- Rentre dans une sockaddr l'adresse (par exemple IPv4) et le port du serveur.

```
struct sockaddr_in sa = { .sin_family = AF_INET,
                          .sin_port = htons(port) };
if (inet_pton(AF_INET, addr_ip, &sa.sin_addr) != 1) { /* ... */ }
socklen_t sl = sizeof(sa);
```

- Puis demande une connexion au serveur.

```
if (connect(sock, (struct sockaddr *) &sa, sl) < 0) {
    perror("connect");
    exit(3);
}
```

- Test de connect non négociable.

Côté serveur : plusieurs *sockets*

- Une *socket* (dite « d'écoute ») pour accepter les demandes de connexions
- et une *socket* par client connecté.
- Le serveur crée une *socket* d'écoute TCP (par exemple IPv4).

```
int sock_l = socket(AF_INET, SOCK_STREAM, 0);
if (sock_l < 0) { /* ... */ }
```

- L'attache à un port local (et éventuellement une adresse locale).

```
struct sockaddr_in sa = { .sin_family = AF_INET,
              .sin_port = htons(PORT_INCP),
              .sin_addr.s_addr = htonl(INADDR_ANY) };
socklen_t sl = sizeof(sa);
/* Optionnel : faire en sorte de pouvoir réutiliser l'adresse sans
 * attendre après la fin du serveur. */
int opt = 1;
setsockopt(sock_l, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(int));
if (bind(sock_l, (struct sockaddr *) &sa, sl) < 0) {
    perror("bind");
    exit(3);
}
```

Côté serveur : appel système `listen`

- puis, une seule fois configure demande au système de configurer sa *socket* comme *socket* d'écoute : appel système `listen`

```
if (listen(sock_l, 128) < 0) { perror("listen"); exit(2); }
```

- test non négotiable (*Port already in use*, ...)
- Le deuxième paramètre de `listen` est historiquement la taille maximale de la file d'attente des demandes de connexion, il est maintenant ignoré par défaut, au moins dans Linux. Mon 128 est une autre valeur par défaut trouvée quelque part...

Côté serveur : appel système `accept`

- Dans une boucle infinie (et sans doute avec des *threads* supplémentaires) :

```
struct sockaddr_in sa_clt;
socklen_t sl = sizeof(sa_clt);
int s = accept(sock_l, (struct sockaddr *) &sa_clt, &sl);
if (s < 0) { /* ... */ }
```

- appel système bloquant en attente de connexion
- en cas de succès, retourne le descripteur de fichier d'une nouvelle *socket* qui ne sert qu'à communiquer avec ce client
- Les paramètres suivants permettent de renseigner la `sockaddr` du client si on veut, par exemple, écrire son adresse IP et son port dans un fichier de *log*. Contrairement à UDP, on n'en a pas besoin pour répondre au client, la *socket* retournée est connectée au client.

Fin de connexion : `close`

- Client ou serveur (sur sa *socket* d'échange avec ce client) peuvent mettre fin à la connexion en fermant (avec `close`) la *socket*.
- Ceci transmet un segment `FIN` à l'autre côté, qui aquittera ce segment `FIN`.
- Au niveau des appels systèmes, les lectures de l'autre côté retourneront 0, donc on peut savoir que l'autre côté a demandé une fin de connexion (et on peut arrêter la lecture).

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

Flux

- Rappel : UDP fournit des datagrammes, courts messages avec un début et une fin. Chaque lecture correspond à la réception d'un datagramme, chaque écriture à l'envoi d'un datagramme.
- TCP fournit un flux (bidirectionnel) d'octets, il n'y a plus de correspondance entre lecture et réception.
- Pour lire des octets reçus, on utilise `read` (ou `recv` si l'on veut ajouter des flags).
- Pour envoyer des octets, on utilise `write` (ou `send` si l'on veut ajouter des flags).

Important : Lecture dans une *socket* TCP

```
char buf[256];
ssize_t n = read(sock, buf, 256);
```

- Appel système bloquant.
- Si le système retourne 0, le processus distant a fermé sa *socket*.
- Si le système retourne $n > 0$, alors n octets ont été lus dans le flux et placés dans `buf` (et $n \leq 256$).
- TCP ne fournit aucun moyen de savoir alors s'il va y avoir d'autres octets à lire dans le flux.
- Il n'y a pas non plus de lien direct entre le nombre d'octets lus dans le flux et le nombre d'octets écrits par le processus distant (à part que l'on ne peut pas lire plus que ce qui a été envoyé).

Important : Lecture dans une *socket* TCP

```
char buf[256];
ssize_t n = read(sock, buf, 256);
```

- Appel système bloquant.
- Si le système retourne 0, le processus distant a fermé sa *socket*.
- Si le système retourne $n > 0$, alors n octets ont été lus dans le flux et placés dans `buf` (et $n \leq 256$).
- TCP ne fournit aucun moyen de savoir alors s'il va y avoir d'autres octets à lire dans le flux.
- Il n'y a pas non plus de lien direct entre le nombre d'octets lus dans le flux et le nombre d'octets écrits par le processus distant (à part que l'on ne peut pas lire plus que ce qui a été envoyé).
- Faut-il encore lire dans la *socket* ?
- Est-ce le moment de répondre quelque-chose ?

Important : Lecture dans une *socket* TCP

```
char buf[256];
ssize_t n = read(sock, buf, 256);
```

- Appel système bloquant.
- Si le système retourne 0, le processus distant a fermé sa *socket*.
- Si le système retourne $n > 0$, alors n octets ont été lus dans le flux et placés dans `buf` (et $n \leq 256$).
- TCP ne fournit aucun moyen de savoir alors s'il va y avoir d'autres octets à lire dans le flux.
- Il n'y a pas non plus de lien direct entre le nombre d'octets lus dans le flux et le nombre d'octets écrits par le processus distant (à part que l'on ne peut pas lire plus que ce qui a été envoyé).
- Faut-il encore lire dans la *socket* ?
- Est-ce le moment de répondre quelque-chose ?
- Seule la connaissance du protocole applicatif permet de répondre à ces questions.

Arrêter de lire ?

Savoir quand il faut s'arrêter de lire devient dès lors un problème difficile.

Arrêter de lire ?

Savoir quand il faut s'arrêter de lire devient dès lors un problème difficile.
Première solution, comme INCP, fixer la taille des messages (4 octets).

Arrêter de lire ?

Savoir quand il faut s'arrêter de lire devient dès lors un problème difficile.
Première solution, comme INCP, fixer la taille des messages (4 octets).
Puis lire tant que le nombre total d'octets lus est inférieur à la taille du message, en mettant les octets lus au bon endroit.

Arrêter de lire ?

Deuxième solution, comme la première version d'HTTP.

- Si on sait que le serveur va fermer la connection (la *socket*) à la fin de son message...

Arrêter de lire ?

Deuxième solution, comme la première version d'HTTP.

- Si on sait que le serveur va fermer la connection (la *socket*) à la fin de son message...
- on peut lire jusqu'à ce que `read` retourne 0.

Arrêter de lire ?

Troisième solution, comme les *requêtes* HTTP, un octet, ou une suite d'octets signale la fin de l'en-tête.

- Une requête HTTP est formée de plusieurs lignes.
- Une ligne vide met fin au message.
- Question : qu'est-ce qu'une ligne ?

Arrêter de lire ?

Troisième solution, comme les *requêtes* HTTP, un octet, ou une suite d'octets signale la fin de l'en-tête.

- Une requête HTTP est formée de plusieurs lignes.
- Une ligne vide met fin au message.
- Question : qu'est-ce qu'une ligne ?
- Fin de ligne sous Unix : `'\n'` (*new line*, *line feed*, LF, ASCII 0A).
- Fin de ligne sous Windows : `'\r'` puis `'\n'` (*carriage return and line feed*, CRLF, ASCII 0D ASCII 0A)

Arrêter de lire ?

Troisième solution, comme les *requêtes* HTTP, un octet, ou une suite d'octets signale la fin de l'en-tête.

- Une requête HTTP est formée de plusieurs lignes.
- Une ligne vide met fin au message.
- Question : qu'est-ce qu'une ligne ?
- Fin de ligne sous Unix : `'\n'` (*new line*, *line feed*, LF, ASCII 0A).
- Fin de ligne sous Windows : `'\r'` puis `'\n'` (*carriage return and line feed*, CRLF, ASCII 0D ASCII 0A)
- Question de l'encodage des caractères également.

Exemple de requête HTTP

fichier `req_perdu_com.txt`

```
GET / HTTP/1.1
Host: perdu.com
Accepted-Language: fr
```

(avec une ligne vide à la fin !).

Exemple de requête HTTP

fichier `req_perdu_com.txt`

```
GET / HTTP/1.1
Host: perdu.com
Accepted-Language: fr
```

(avec une ligne vide à la fin !).
Commande `socat`

```
$ socat TCP4:www.perdu.com:80,crlf <req_perdu_com
```

Exemple de requête HTTP

fichier `req_perdu_com.txt`

```
GET / HTTP/1.1
Host: perdu.com
Accepted-Language: fr
```

(avec une ligne vide à la fin !).
Commande `socat`

```
$ socat TCP4:www.perdu.com:80,crlf <req_perdu_com
```

Réponse serveur :

```
HTTP/1.1 200 OK
Date: Thu, 07 Apr 2022 08:58:06 GMT
Server: Apache
...
* <----- vous &ecirc;tes ici</pre></strong></body></html>
```

Exemple de requête HTTP

fichier `req_perdu_com.txt`

```
GET / HTTP/1.1
Host: perdu.com
Accepted-Language: fr
```

(avec une ligne vide à la fin !).
Commande `socat`

```
$ socat TCP4:www.perdu.com:80,crlf <req_perdu_com
```

Réponse serveur :

```
HTTP/1.1 200 OK
Date: Thu, 07 Apr 2022 08:58:06 GMT
Server: Apache
...
* <----- vous &ecirc;tes ici</pre></strong></body></html>
```

Le serveur de `perdu.com` a fermé la connection.

Arrêter de lire ?

Dernière solution : mettre quelque part dans le message la taille du message (ou d'une partie du message).
Encore avec HTTP :

- les réponses serveur incluent une en-tête ;
- qui se termine encore par une ligne vide.
- l'en-tête contient une ligne (par exemple)

  `Content-Length: 218`

  qui indique le nombre d'octets du corps du message.

Une fois que j'ai fini de lire l'en-tête (comment le savoir ?) je sais que je dois lire `218` octets.

Alignement en mémoire et remplissage

Sérialisation et désérialisation

Rôle de TCP

sockets TCP : connexion et fin de connexion

sockets TCP : lectures et écritures

Attendre qu'un fichier soit prêt : appel système `poll`

Raison d'être de poll

- Que faire si l'on veut lire les lignes d'entrées du terminal et dans un tube ?
- On ne peut pas faire la chose suivante :

```
ssize_t n1, n2;
for (;;) {
  n1 = read(0, buf1, n1);
  /* faire des choses avec buf1, n1 */
  n2 = read(tube[0], buf2, n2);
  /* faire des choses avec buf2, n2 */
}
```

- Ça ne marcherait qu'à la condition, très improbable, qu'il y ait toujours alternativement une ligne à lire dans le terminal, puis des octets à lire dans le tube.
- On voudrait attendre que le terminal ou le tube soit prêt pour la lecture, et savoir lequel des deux (ou les deux, le cas échéant).

poll

- NOM
  poll - Attendre un événement concernant un descripteur de fichier
- ```
  #include <poll.h>
  int poll(struct pollfd *fds, nfds_t nfds, int timeout);
  struct pollfd {
     int   fd;         /* Descripteur de fichier */
     short events;     /* Événements attendus    */
     short revents;    /* Événements détectés    */
  };
  ```
- On donne un tableau de `nfds` `struct pollfd` et un `timeout` (temps maximal d'attente en millisecondes) qui, s'il vaut -1, est infini. L'appel `poll` est, bien sûr, bloquant.
- Chaque `struct pollfd` contient le descripteur de fichier en question, des `bits events` pour les événements attendus.
- Lorsque `poll` retourne, les bits de `revents` (événements détectés) sont renseignés : c'est eux qu'il faut tester pour savoir ce qui a réveillé `poll`.
- Valeur de retour : 0 si le timeout a expiré, -1 en cas d'erreur (`errno` positionné), sinon, le nombre d'éléments de `pollfds` pour lesquels `revents` a des bits positionnés.
  ```

Événements de `poll`

Les différents bits de `events` qui nous intéressent sont

- `POLLIN` : il y a des octets à lire
- `POLLOUT` : l'écriture est maintenant possible (pour écrire dans des tubes ou des *socket* TCP)

En plus, dans `revents` (mais pas dans `events`) :

- `POLLERR` : erreur, en particulier pour une extrémité d'écriture de tube, lorsque l'extrémité de lecture a été fermée
- `POLLHUP` : (pour *hang up*, raccrocher) plantage, et notamment pour l'extrémité de lecture d'un tube ou pour une socket, peut indiquer que le pair a fermé son extrémité de communication (mais il peut rester des données à lire dans le tube ou la socket).
- `POLLNVAL` : (pour non valable), `fd` n'est pas ouvert.

On s'intéresse surtout à `POLLIN` et `POLLHUP` dans le cadre de ce cours.

Exemple avec 2 tubes nommés

Dans l'exemple qui va suivre :

- On crée préalablement deux tubes nommés `tube1` et `tube2` dans le répertoire courant. Deux processus écrivent, respectivement, dans `tube1` et `tube2` ; ici, simplement dans deux terminaux différents :

  ```
  $ stdbuf -oL cat >tube1
  $ stdbuf -oL cat >tube2
  ```

- Pour l'instant, les deux processus sont bloqués dans l'ouverture des tubes, qui ne sont pas encore ouverts en lecture.
- On utilise la commande `stdbuf` pour forcer la sortie à être « *line buffered* ». Sinon, par défaut, la sortie dans un fichier qui n'est pas le terminal est « *full buffered* » et les écritures n'ont lieu que lorsque le buffer est plein (ou le fichier fermé).
- Un troisième programme `exple_poll`, ouvre en lecture ces deux tubes, fait un `poll` pour attendre sur les deux tubes (et recopie bêtement sur le terminal ce qu'il a lu).

Exemple de programme avec poll

```
int f1, f2;
if ((f1 = open("tube1", O_RDONLY)) < 0) { /* ... */ }
if ((f2 = open("tube2", O_RDONLY)) < 0) { /* ... */ }
struct pollfd fds[2] = {
  { .fd = f1, .events = POLLIN },
  { .fd = f2, .events = POLLIN }};
char buff[2048];
ssize_t n;
int nb_ouverts = 2;
while (nb_ouverts > 0 && poll(fds, 2, -1) > 0) {
  if (fds[0].revents & (POLLIN | POLLHUP)) {
    if ((n = read(f1, buff, sizeof(buff))) == 0) {
      printf("tube1 fermé en écriture\n");
      close(f1); nb_ouverts--;
    } else {
      printf("Données reçues depuis tube1 :\n");
      write(1, buff, n);
    }
  }
  if (fds[1].revents & (POLLIN | POLLHUP)) {
    /* comme au-dessus avec f2 et tube2 */
  }
}
```

POLLIN | POLLHUP

- Malheureusement, sur certains systèmes, dont Linux, `POLLIN` n'est mis à 1 que lorsqu'il y a vraiment des données à lire (au lieu de signaler qu'un `read` serait non bloquant et sans erreur).
- Ce qui peut faire rater la fin du fichier.
- Il faut donc tester les deux bits `POLLIN | POLLHUP`, puis `read` et faire ce qu'il y a à faire si ce `read` donne `0`.
- Plus de détails sur cette page : `https://www.chiark.greenend.org.uk/greenend.org.uk/rjk/tech/poll.html`